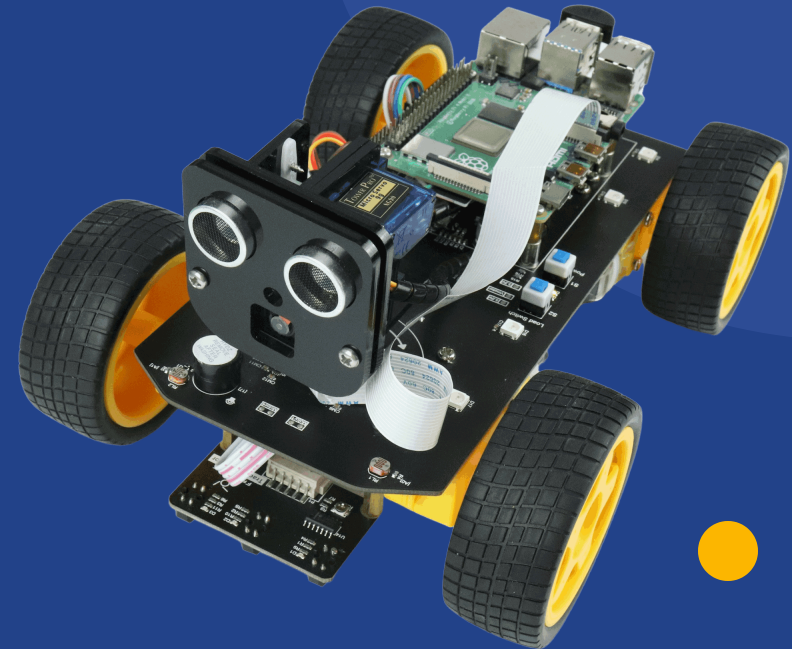




LESSON 2 – SERVER & CLIENT COMMANDS

Machine Learning with Raspberry Pi & Smart Car

Today the car drives. We spin all four wheels, beep the buzzer – and build the command strings the car listens for.



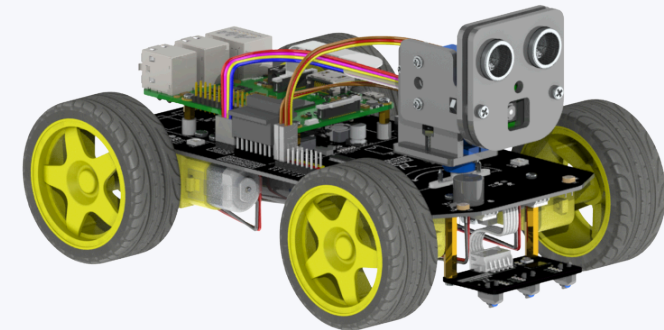


LESSON 2 – SERVER & CLIENT COMMANDS

From parts to commands

Last time we drove the parts directly. Today we add the wheels – and the idea of a command.

- **Last time:** lit the LEDs, swept the servo head, and read the ultrasonic sensor.
- **We learned the shape:** `setup() → work → destroy()`, with `import car_setup` first.
- **Today:** keep that shape, add the **wheels**, the **buzzer**, and the idea of **commands**.





LESSON 2 – SERVER & CLIENT COMMANDS

Today's goal: a command demo

One program that drives, beeps, and flashes – and prints the exact command each time.

- 1 **Drives forward, back, left, right, then stops**
- 2 **Beeps the buzzer and flashes the LEDs blue**
- ★ **Prints the command string the car would receive**



The printed command is the real lesson – not just the driving.

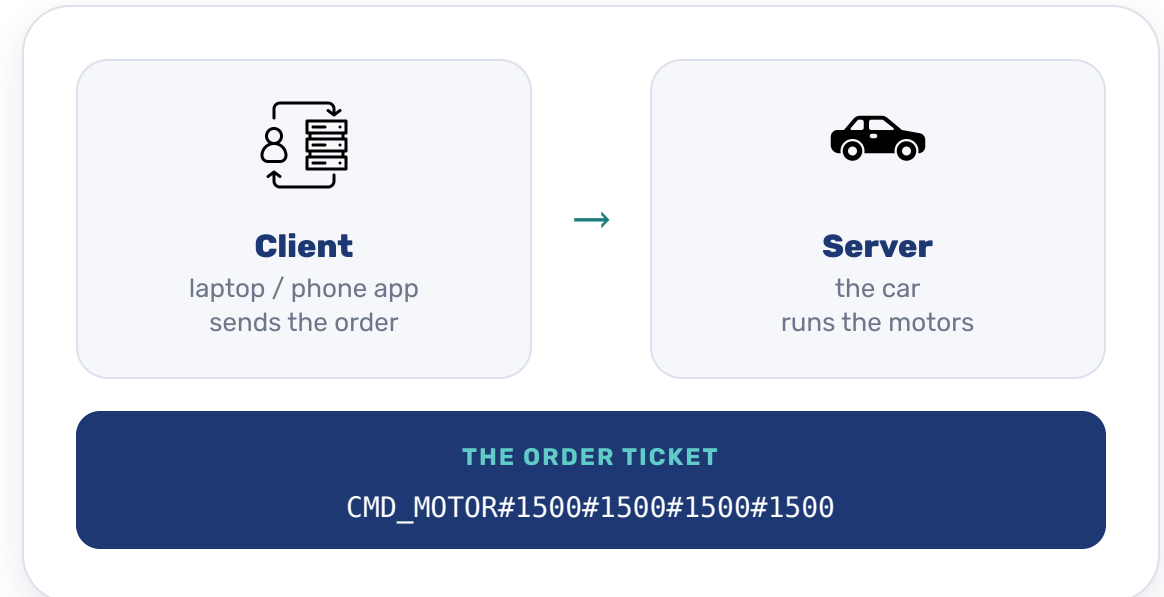


LESSON 2 – SERVER & CLIENT COMMANDS

Big idea: a client and a server

An app sends orders; the car carries them out. Today we play both roles in one file.

- **Client** – an app on your laptop or phone – sends an order like *"drive forward"*.
- **Server** – the car – **does** it with the motors.
- Think of it like a restaurant: the client **orders**, the server (kitchen) **cooks**.
- Today both roles live in one file: we **build the order**, then **run it** on the car.





LESSON 2 – SERVER & CLIENT COMMANDS

Commands are just text

The client and server talk with short strings, glued together with `#`.

- Nothing magic travels over the wire – it's **only a string**.
- A **command name** comes first, then its values.
- The pieces are joined with a `#` between each one.
- We'll write a tiny helper that builds these so we can **see** the message.

CMD_MOTOR#1500#1500#1500#1500

set the four wheels to 1500

CMD_BUZZER#1

buzzer on

CMD_LED#255#0#0#255

set the LEDs to a color

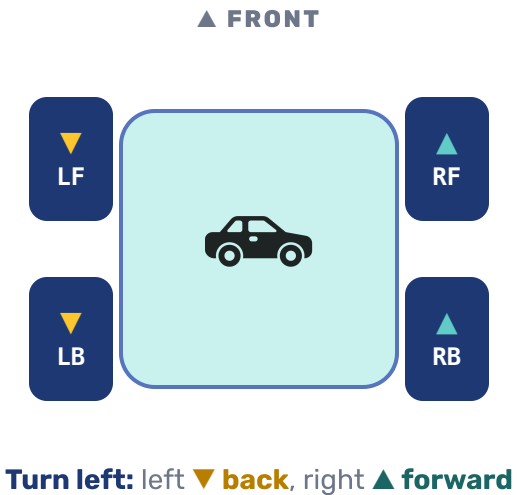


LESSON 2 – SERVER & CLIENT COMMANDS

Four wheels, four numbers

Always in the same order – and turning is just a trick with the signs.

- Order is always **left-front, left-back, right-front, right-back**.
- `0` = stopped ... `4095` = full speed. **Negative** = backward.
- **Turning trick:** spin one side backward, the other side forward.
- `1500` is a gentle, safe classroom speed.





LESSON 2 – SERVER & CLIENT COMMANDS • THE FILE, CHUNK 1

Start the file: imports

`car_setup` first, then the parts we control and the command names.

- `import car_setup` first – it finds the car's modules.
- Then the four **parts** we drive today, plus the command names.
- Watch the casing: lowercase files, but `COMMAND` is all-caps.

```
lesson_2_server_client_commands.py

"""Lesson 2: Server and Client."""

import car_setup      # add the car's code folders
import time

from motor import Motor    # the four driving wheels
from buzzer import Buzzer  # the beeper
from led import Led        # the LED lights
from command import COMMAND # the command names
```



LESSON 2 – SERVER & CLIENT COMMANDS • THE FILE, CHUNK 2

Settings: the speed and the moves

Each move is four wheel values in LF / LB / RF / RB order.

- **Forward** = all positive • **Backward** = all negative.
- **Turns** reverse one side • **Stop** = all zero.
- Look at `TURN_LEFT` – left wheels negative, right positive.

```
lesson_2_server_client_commands.py
```

```
# ---- settings you can change ----  
DRIVE_SPEED = 1500          # 0 = stop ... 4095 = full speed  
  
# wheel order: left-front, left-back, right-front, right-back  
FORWARD = (DRIVE_SPEED, DRIVE_SPEED, DRIVE_SPEED, DRIVE_SPEED)  
BACKWARD = (-DRIVE_SPEED, -DRIVE_SPEED, -DRIVE_SPEED, -DRIVE_SPEED)  
TURN_LEFT = (-DRIVE_SPEED, -DRIVE_SPEED, DRIVE_SPEED, DRIVE_SPEED)  
TURN_RIGHT = (DRIVE_SPEED, DRIVE_SPEED, -DRIVE_SPEED, -DRIVE_SPEED)  
STOP = (0, 0, 0, 0)  
  
LED_ALL = 0xFF              # all eight LEDs at once
```



LESSON 2 – SERVER & CLIENT COMMANDS • THE FILE, CHUNK 3

Name the parts, then setup()

Same pattern as Lesson 1 – set names to `None`, then build them in one place.

- Name the parts now (`None`), build them in `setup()` .
- `global` means "use the names above – don't make new ones."
- `cmd = COMMAND()` gives us names like `cmd.CMD_MOTOR` .

```
lesson_2_server_client_commands.py

# ---- car parts (created in setup) ----
motor = None
buzzer = None
led = None
cmd = None

def setup():
    """Create the parts and command helper."""
    global motor, buzzer, led, cmd
    motor = Motor()
    buzzer = Buzzer()
    led = Led()
    cmd = COMMAND()           # names like cmd.CMD_MOTOR
```



LESSON 2 – SERVER & CLIENT COMMANDS • THE FILE, CHUNK 4

Build the command string yourself

This is the heart of the lesson: glue a name and its values with `#`.

```
["CMD_MOTOR", 1500, 1500]
```

↓ **join with "#"**

```
CMD_MOTOR#1500#1500
```

- Start with the name, add each value as **text**.
- `"#".join(parts)` puts a `#` between every piece.

```
lesson_2_server_client_commands.py

def format_command(name, values):
    """Build CMD_MOTOR#1500#1500."""
    parts = [name]           # start with the name
    for value in values:     # add each value as text
        parts.append(str(value))
    return "#".join(parts)   # glue with "#"
```



LESSON 2 – SERVER & CLIENT COMMANDS • THE FILE, CHUNK 5

drive(): print, then move

Show the command first – then actually turn the wheels.

- First `print` the command so students **see** it.
- Then `setMotorModel(lf, lb, rf, rb)` drives the four wheels.
- Connect the printed line to the wheels turning – **message** → **action**.

```
lesson_2_server_client_commands.py

def drive(move, seconds):
    """Run one move, printing its command."""
    print("Sending:", format_command(cmd.CMD_MOTOR, move))
    motor.setMotorModel(move[0], move[1], move[2], move[3])
    time.sleep(seconds)
```



LESSON 2 – SERVER & CLIENT COMMANDS • THE FILE, CHUNK 6

test_buzzer(): beep on, then off

The buzzer uses strings – "1" for on, "0" for off.

HEADS UP

Pass the **string** "1", not the number 1.



buzzer.run("1") – correct.



buzzer.run(1) – a common silent bug.

```
lesson_2_server_client_commands.py

def test_buzzer():
    """Beep on, then off."""
    print("Sending:", format_command(cmd.CMD_BUZZER, ["1"]))
    buzzer.run("1")          # "1" = on, "0" = off (strings!)
    time.sleep(1)
    buzzer.run("0")
```



LESSON 2 – SERVER & CLIENT COMMANDS • THE FILE, CHUNK 7

test_leds(): flash blue

Reuse `ledIndex(which, r, g, b)` from Lesson 1 – all LEDs blue, then off.



`(0, 0, 255)` = no red, no green, full blue → **blue**, then off.

```
lesson_2_server_client_commands.py

def test_leds():
    """Turn every LED blue, then off."""
    print("Sending:", format_command(cmd.CMD_LED, [LED_ALL, 0, 0, 255]))
    led.ledIndex(LED_ALL, 0, 0, 255)    # blue
    time.sleep(1)
    led.ledIndex(LED_ALL, 0, 0, 0)      # off
```



LESSON 2 – SERVER & CLIENT COMMANDS • THE FILE, CHUNK 8

loop(): the whole routine

Every move in order, then beep, then flash – once.

- Run each move, then the buzzer and LED tests.
- We `drive(STOP, 1)` **before** the tests – no rolling during the beep.
- Each helper still prints its own `Sending:` line.

```
lesson_2_server_client_commands.py

def loop():
    """Drive, beep, then flash – once."""
    drive(FORWARD, 1)
    drive(BACKWARD, 1)
    drive(TURN_LEFT, 1)
    drive(TURN_RIGHT, 1)
    drive(STOP, 1)          # stop before the tests
    test_buzzer()
    test_leds()
```



LESSON 2 – SERVER & CLIENT COMMANDS • THE FILE, CHUNK 9

Clean up: destroy()

Always finish stopped and quiet – motors off, buzzer off, LEDs off.

- Stopping the motors here is a **safety must**.
- A crashed program should never leave the wheels spinning.
- Guard each part with `is not None` in case setup failed.

```
lesson_2_server_client_commands.py

def destroy():
    """Leave the car stopped and quiet."""
    if motor is not None:
        motor.setMotorModel(0, 0, 0, 0) # stop wheels
    if buzzer is not None:
        buzzer.run("0") # silence
    if led is not None:
        led.ledIndex(LED_ALL, 0, 0, 0) # LEDs off
```



LESSON 2 – SERVER & CLIENT COMMANDS • THE FILE, CHUNK 10

The main block ties it together

Same safe pattern as Lesson 1 – try, except, finally.

- **try** runs `setup()` then `loop()`.
- **except** catches **Ctrl+C** – no scary error.
- **finally always** runs `destroy()` – even after a crash.

```
lesson_2_server_client_commands.py

if __name__ == "__main__":
    try:
        setup()
        loop()
    except KeyboardInterrupt:
        pass                # Ctrl+C: stop quietly
    finally:
        destroy()           # always leave the car safe
```



LESSON 2 – SERVER & CLIENT COMMANDS

You built the whole file

It both builds the commands and carries them out.



The same **setup** → **work** → **destroy** skeleton as Lesson 1, plus the command idea.



Builds the message, then acts on it.



LESSON 2 – SERVER & CLIENT COMMANDS

Run it – wheels off the ground!

Lift the car for the first run, then run it from the Pi's terminal with `sudo`.

**SAFETY FIRST**

Put a book or box under the car so the wheels spin freely on the first run.

- Watch the terminal – each action prints its `Sending:` line.
- Then put it on the floor for a supervised second run.

Pi terminal

```
cd Smart-Car-Level-3/Code/User  
sudo python3 lesson_2_server_client_commands.py
```

output

```
Sending: CMD_MOTOR#1500#1500#1500#1500  
Sending: CMD_MOTOR#-1500#-1500#-1500#-1500  
Sending: CMD_BUZZER#1  
Sending: CMD_LED#255#0#0#255
```



LESSON 2 – SERVER & CLIENT COMMANDS

Try it yourself

Tune the demo and make the car your own.

- Lower `DRIVE_SPEED` to `1000` (gentler), or raise it carefully.
- Add a move like `SPIN = (DRIVE_SPEED, DRIVE_SPEED, -DRIVE_SPEED, -DRIVE_SPEED)` and call it in `loop()`.
- Change the LED color in `test_leds()`.

CHALLENGE

Drive a square

Make the car trace a square: forward, then turn – four times. Tuning the turn timing teaches a lot.



LESSON 2 – SERVER & CLIENT COMMANDS

Common issues

If something's off, it's almost always one of these.



Car drives off the table

Lift the wheels first; lower `DRIVE_SPEED`.



Wheels weak or uneven

Usually a low battery, or a loose motor wire.



Buzzer is silent

You passed `1` instead of the string `"1"`.



ModuleNotFoundError

Casing, or `car_setup` not imported first.



Check the battery

Weak or uneven driving is almost always low power. Keep charged spares ready.



LESSON 2 – SERVER & CLIENT COMMANDS

You made it move!

You built command strings and drove the car with them.

- ✓ Learned the client → command → server idea
- ✓ Built command strings yourself with `format_command`
- ✓ Drove all four wheels, beeped, and flashed the LEDs

UP NEXT

Lesson 3 Sensors & multithreading

Read sensors with the ADC, and run a battery watchdog in the background while the car does something else.



• COACH ONLY • HIDDEN SLIDE

End-of-class checklist

Don't present this slide. Run through it right after class.

1

Post the after-class recap

Summarise what you covered on the class page so students can catch up.

2

Log absent students

Post anyone who missed class in the **#attendance** Discord channel.

3

Charge for next class

Charge the cars before Lesson 3 – low battery breaks the driving demos.

EVERY CLASS

✓ Post a class announcement

✓ Introductions if needed

✓ Note absences in #attendance